



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina
Análise Numérica**

Relatório de implementações realizadas

por Reinan Lopes Argolo

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilhéus – BA

2026

Sumário

1	Linguagem Escolhida e Justificativas	4
2	Método da Bissecção	5
2.1	Estratégia de Implementação	5
2.2	Estrutura dos Arquivos de Entrada e Saída	6
2.3	Problema Teste 1: Tempo de Subida de um Amplificador	6
2.4	Análise de Convergência	7
2.5	Dificuldades Enfrentadas	8
3	Método 2	10
3.1	Estratégia de Implementação	10
3.2	Estrutura dos Arquivos de Entrada/Saída	10
3.3	Problema teste 1, 2, 3...	10
3.4	Dificuldades enfrentadas	10
4	Considerações Finais	11

Lista de Figuras

1	Gráfico de convergência montado automaticamente via TikZ/pgfplots buscando dados do CSV do ambiente	7
2	Gráfico de convergência do erro relativo e magnitude da função $f_1(T)$ pelo Método da Bissecção.	8

Lista de Tabelas

1 Histórico iterativo do Método da Bissecção para a função $f_1(T)$ (Busca
de $T_{0.1}$). 7

1 Linguagem Escolhida e Justificativas

Para o desenvolvimento dos algoritmos e a execução das rotinas computacionais propostas neste relatório de métodos numéricos, optou-se pela utilização da linguagem de programação Python. A escolha dessa ferramenta justifica-se, pela sua expressiva adoção na comunidade científica e pela vasta disponibilidade de bibliotecas para manipulação matemática e computação científica [3].

Entre as principais extensões que fundamentam essa decisão, destaca-se a biblioteca *NumPy*, a qual fornece suporte nativo e eficiente para o processamento de arranjos e matrizes multidimensionais. Essa biblioteca implementa operações de álgebra linear com rotinas de baixo nível, garantindo o alto desempenho exigido nos cálculos numéricos iterativos [1]. Em complemento, o uso do ecossistema *SciPy* oferece um conjunto de algoritmos fundamentais para interpolação, integração numérica, otimização e resolução de sistemas lineares e não lineares [4].

Ademais, Python caracteriza-se por uma sintaxe de alto nível e semântica clara, o que favorece a legibilidade do código e permite a prototipagem rápida de abstrações matemáticas [2]. Dessa forma, a redução da complexidade de implementação viabiliza um foco maior na análise de convergência, na estabilidade dos métodos e na mitigação de erros de truncamento e arredondamento, requisitos indispensáveis para a consistência deste trabalho acadêmico.

2 Método da Bissecção

O Método da Bissecção constitui uma das abordagens numéricas mais elementares e robustas para a determinação de raízes de funções reais. Fundamentado no Teorema de Bolzano, o algoritmo garante convergência incondicional, desde que a função seja contínua no intervalo fechado $[a, b]$ e que os valores da função nos extremos apresentem sinais opostos.

2.1 Estratégia de Implementação

O desenvolvimento do algoritmo foi estruturado de forma modular, sendo sua implementação principal em `src/metodos/zeros_de_funcao.py`.

Código 1: Código de implementação do Método da Bissecção

```
1
2 def bisseccao(funcao, a, b, tol=1e-6, max_iter=100):
3     historico = []
4
5     for i in range(max_iter):
6         c = (a + b)/2
7         fc = funcao(c)
8         denominador = a + b
9
10        if denominador != 0:
11            er = abs((a - b) / denominador)
12        else:
13            er = abs(b - a)
14
15        historico.append({
16            'iter': i,
17            'a': a,
18            'b': b,
19            'c': c,
20            'f(c)': fc,
21            'erro': er
22        })
23
24        if fc == 0 or er < tol:
25            return c, i, historico
26
27        if utils.teorema_bolzano(funcao, a, c):
28            b = c
29        else:
30            a = c
31
32    return c, max_iter, historico
```

A cada iteração k , o ponto médio do subintervalo vigente é calculado pela média aritmética simples:

$$c_k = \frac{a_k + b_k}{2} \quad (1)$$

A seleção do subintervalo subsequente é definida pela comparação dos valores de a e c , se eles não foram diferentes, então o novo intervalo será $[c, b]$, caso sejam

diferentes, então $[a, c]$

No que tange à avaliação da precisão, adotou-se preliminarmente o erro relativo associado à amplitude do intervalo, definido matematicamente por:

$$\epsilon_{rel} = \left| \frac{a_k - b_k}{a_k + b_k} \right|. \quad (2)$$

Caso a soma de $a_k + b_k$ seja igual a zero, assume-se apenas que $\epsilon_{abs} = |b_k - a_k|$.

Para limitar o algoritmo e evitar que exista um processamento desnecessário, estipulou-se um número máximo de iterações (max_iter) e uma tolerância (tol) para a convergência. O critério de parada é acionado quando a função avaliada no ponto médio c_k for exatamente zero ou quando o erro relativo for inferior à tolerância estabelecida.

2.2 Estrutura dos Arquivos de Entrada e Saída

Sendo este um critério de avaliação, todos os dados de entrada e saída foram organizados em uma estrutura de arquivos padronizada, visando a clareza, a reprodutibilidade e a facilidade de análise posterior.

Os arquivos foram organizados em uma hierarquia de diretórios, onde os dados de entrada foram alocados em `dados/entradas/` e os resultados processados foram persistidos em `dados/saidas/`.

Os dados são salvos em arquivos de texto simples (TXT) para as entradas, contendo os limites do intervalo, a tolerância e também a quantidade máxima de iterações. Já os resultados das iterações são armazenados em arquivos CSV, contendo colunas para o número da iteração, os valores de a , b , c , $f(c)$ e o erro relativo, facilitando a análise quantitativa e a geração de gráficos subsequentes.

2.3 Problema Teste 1: Tempo de Subida de um Amplificador

O problema proposto requer a determinação do tempo de subida de um amplificador eletrônico, definido pelo intervalo temporal necessário para que a resposta $g(T)$ evolua de 10% a 90% de seu valor assintótico. A função normalizada é expressa por:

$$g(T) = 1 - \left(1 + T + \frac{T^2}{2} \right) e^{-T}. \quad (3)$$

A resolução demanda a determinação de duas raízes, $T_{0.1}$ e $T_{0.9}$, as quais satisfazem, respectivamente, as funções auxiliares:

$$f_1(T) = 0.9 - \left(1 + T + \frac{T^2}{2} \right) e^{-T} = 0, \quad (4)$$

$$f_2(T) = 0.1 - \left(1 + T + \frac{T^2}{2} \right) e^{-T} = 0. \quad (5)$$

A Tabela 1 sumariza o histórico iterativo gerado pelo algoritmo implementado para a determinação de $T_{0.1}$ (resolução da Equação 4), partindo do intervalo inicial $[1.0, 2.0]$.

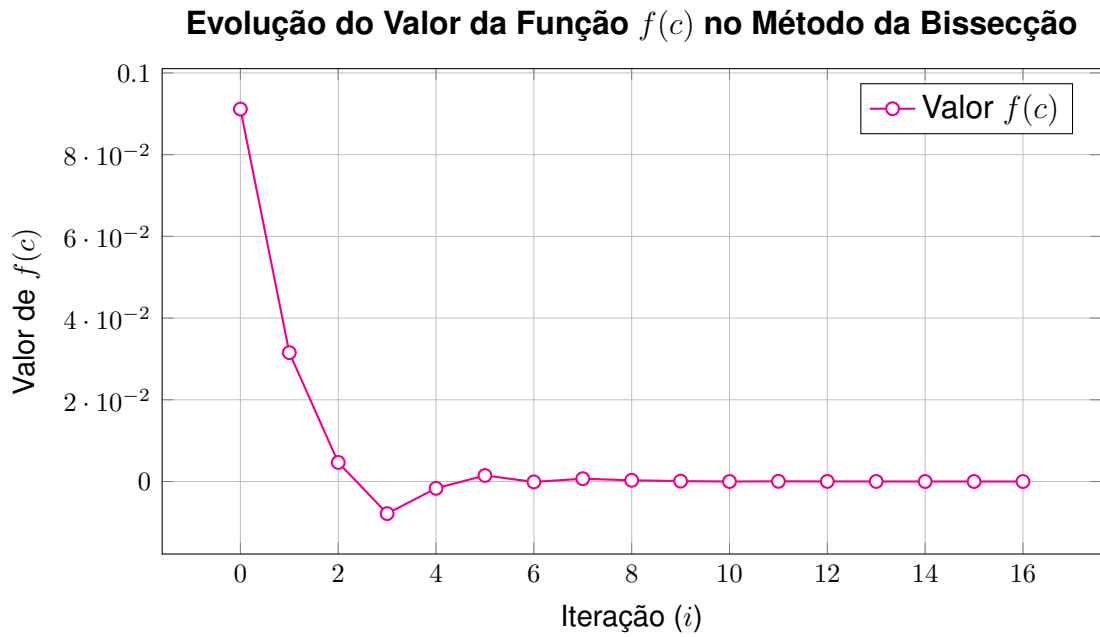


Figura 1: Gráfico de convergência montado automaticamente via TikZ/pgfplots buscando dados do CSV do ambiente

Tabela 1: Histórico iterativo do Método da Bissecção para a função $f_1(T)$ (Busca de $T_{0.1}$).

Iteração	a	b	c	$f_1(c)$	Erro Relativo
0	1.000000	2.000000	1.500000	0.09115317	0.33333333
1	1.000000	1.500000	1.250000	0.03153233	0.20000000
2	1.000000	1.250000	1.125000	0.00466937	0.11111111
3	1.000000	1.125000	1.062500	-0.00785071	0.05882353
4	1.062500	1.125000	1.093750	-0.00167172	0.02857143
5	1.093750	1.125000	1.109375	0.00147894	0.01408451
⋮	⋮	⋮	⋮	⋮	⋮
14	1.102051	1.102112	1.102081	0.00000322	0.00002769
15	1.102051	1.102081	1.102066	0.00000014	0.00001385
16	1.102051	1.102066	1.102058	-0.00000140	0.00000692

Conforme os dados processados, a convergência para $T_{0.1}$ atingiu a tolerância estabelecida na 16ª iteração, resultando em $T_{0.1} \approx 1.102058$. Por procedimento análogo para a Equação 5, determina-se o valor de $T_{0.9} \approx 5.322320$. Consequentemente, o tempo de subida normalizado é dado por:

$$\Delta T = T_{0.9} - T_{0.1} \approx 5.322320 - 1.102058 = 4.220262. \quad (6)$$

2.4 Análise de Convergência

Para ilustrar de forma gráfica a eficácia e o comportamento analítico da técnica frente à função $f_1(T)$, construiu-se a plotagem da convergência evidenciada pela contração progressiva do hipervolume de busca, expressa pelo amortecimento contínuo do erro e da magnitude absoluta da função, ilustrada na Figura 2.

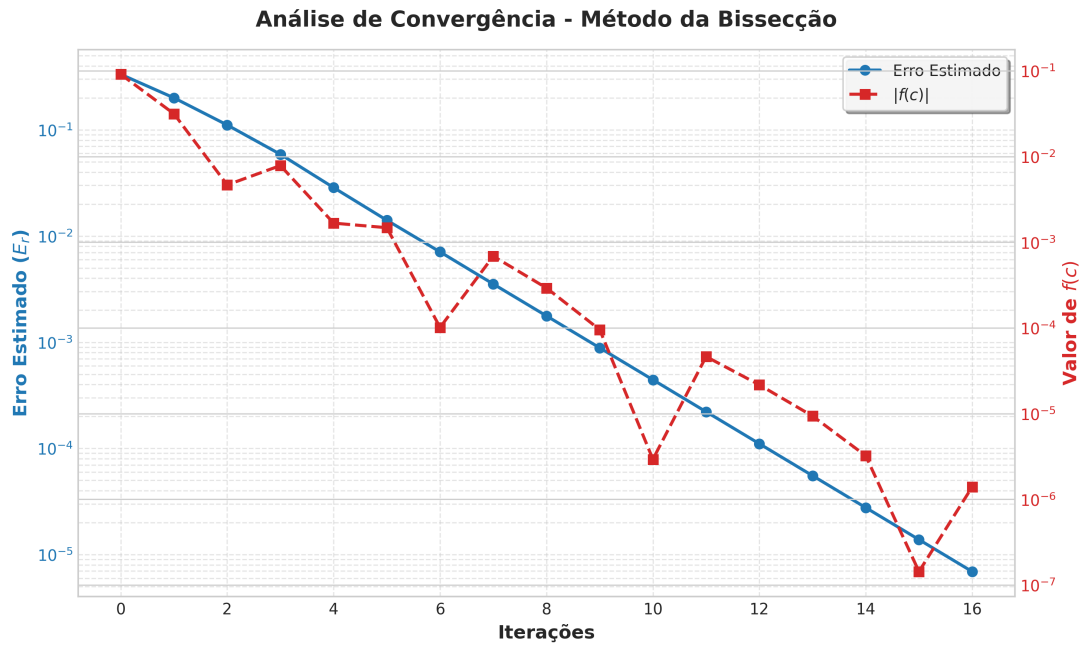


Figura 2: Gráfico de convergência do erro relativo e magnitude da função $f_1(T)$ pelo Método da Bissecção.

Pela análise do gráfico submetido, nota-se que o decréscimo do valor absoluto de $f_1(c)$ possui uma tendência assintótica global de convergência sob o domínio semi-logarítmico, o que corrobora o rigor matemático do algoritmo, atestando sua estabilidade perante o teorema do valor intermediário, reduzindo o erro sempre em $\approx 50\%$ por iteração — um indicativo do característico desempenho de contração linear $\mathcal{O}(N)$ inerente ao Método da Bissecção.

2.5 Dificuldades Enfrentadas

Durante o ciclo de desenvolvimento computacional, dois obstáculos de ordem técnica e arquitetural requisitaram intervenções precisas:

Primeiramente, a formulação matemática para o cálculo de erro incorria em um risco inerente de singularidade. Ao calcular o erro relativo ponderado pela soma dos limites do intervalo $(a_k + b_k)$, constatou-se que caso a raiz estivesse centrada na origem de um intervalo simétrico (i.e., $a_k = -b_k$), o denominador se anulava, deflagrando uma interrupção por `ZeroDivisionError` no interpretador Python. A solução arquitetada consistiu na implementação de um fluxo condicional que, ao detectar a proximidade do denominador à nulidade, transita o cálculo para o domínio do erro absoluto, preservando a estabilidade do algoritmo sem sacrificar o rigor da convergência.

Em segundo lugar, o estabelecimento de uma taxonomia de software aderente aos princípios de responsabilidade única (SOLID) exigiu esforço estrutural. O confinamento da lógica puramente matemática no método iterativo impôs restrições sobre como os dados seriam persistidos. A resolução demandou o encapsulamento do estado de cada iteração em dicionários, agrupados em uma lista transferida para a camada de controle ao fim do processo. Embora tal abstração tenha acrescido complexidade inicial ao desenvolvimento, ela resultou na plena ortogonalidade entre a computação analítica e a persistência em arquivos CSV, promovendo um design de

software altamente escalável.

3 Método 2

3.1 Estratégia de Implementação

Descreva as escolhas que teve que fazer para poder superar eventuais dificuldades com limitações da linguagem, do ambiente de programação, etc. De que forma resolveu tratar os erros, critérios de parada, formato de saída e quaisquer outros fatores importantes para a solução dos problemas.

3.2 Estrutura dos Arquivos de Entrada/Saída

Descreva o formato dos arquivos de entrada e saída e justifique as escolhas.

3.3 Problema teste 1, 2, 3...

Use problemas das seções “Problemas Aplicados a Projetos” (ao final de cada capítulo) do livro Cálculo Numérico, de Neide Franco, para demonstrar que a sua implementação funciona e traz resultados da forma mais eficiente que lhe foi possível. Use, ao menos 3 problemas. Mostre os resultados e teste se eles realmente são a solução. Se não funcionar e você não entender o porquê, explique o que tentou e faça hipóteses.

3.4 Dificuldades enfrentadas

(Descreva as dificuldades aqui)

4 Considerações Finais

(Escreva suas considerações finais aqui)

Referências

- [1] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [2] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib*. Apress, 2 edition, 2018.
- [3] Travis E Oliphant. Python for scientific computing. *Computing in Science & Engineering*, 9(3):10–20, 2007.
- [4] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.